

Self-Surprisal Hard Prompt Compression

Anonymous Authors

Abstract

Hard prompt compression methods such as LLMLingua, SelectiveContext, and LLMLingua-2 rely on a smaller proxy model to score token importance via perplexity or self-information. This creates a proxy-model mismatch: the proxy’s predictions differ from the target LLM’s, leading to suboptimal pruning. We propose Self-Surprisal Hard Prompt Compression (SSHPC), which uses the target LLM’s own next-token logits to compute per-token surprisal—the negative log-likelihood of each token given its preceding context—and prunes the lowest-surprisal tokens. Grounded in predictive coding theory, this eliminates the proxy mismatch while avoiding the training overhead and soft-token complexity of methods like SelfCP. We evaluate SSHPC on four long-context benchmarks using Llama-3-8B across compression ratios from $2\times$ to $10\times$. SSHPC Base outperforms proxy baselines by several percentage points at high compression. Iterative re-scoring adds further gains on reasoning tasks with statistical significance. Latency break-even occurs at $2\times$ compression. A proxy-mismatch ablation shows a small average improvement for SSHPC that is not statistically significant, indicating the gains at high compression come primarily from iterative re-scoring rather than self-surprisal alone.

1 Introduction

Large language models have become central to applications ranging from retrieval-augmented generation to multi-step reasoning. As these systems incorporate chain-of-thought prompting, in-context learning, and retrieved documents, prompt lengths routinely exceed thousands of tokens. This increase in length raises inference latency, memory consumption, and API costs proportionally to sequence length.

Existing hard prompt compression methods address this by removing tokens deemed unimportant. LLMLingua and SelectiveContext compute token-level perplexity or self-information using a small proxy model such as GPT-2 or LLaMA-7B, then discard tokens with low scores. LLMLingua-2 distills a classifier from a proxy model’s outputs. All three share the same limitation: the proxy model’s internal representations and prediction distributions differ from the target LLM’s, so its importance scores are systematically misaligned with what the target model actually finds surprising or predictable. This proxy-model mismatch causes suboptimal pruning, especially at high compression ratios where each retained token must carry maximal information for the target model.

Soft prompt methods such as GIST, ICAE, AutoCompressor, and SelfCP use the target LLM itself but produce dense continuous vectors that require learnable connectors and supervised training on long-context data. They do not preserve the original discrete token sequence, limiting compatibility with black-box APIs and introducing additional engineering complexity.

Why has self-surprisal not been used for hard pruning? Two practical barriers exist. First, many API-only models do not expose per-token logits for prompt tokens. Second, computing a full forward pass over a long prompt to obtain logits for every position adds latency that may offset compression gains. Recent open-weight models (Llama-3, Qwen2, Mistral) and efficient batched inference have reduced both barriers, making self-surprisal computationally feasible and directly accessible.

We propose Self-Surprisal Hard Prompt Compression (SSHPC), which scores tokens by the target LLM’s own next-token surprisal and prunes the lowest-surprisal tokens. At high compression ratios ($\geq 8\times$), iterative re-scoring adapts to the changed context. This uses the target model’s own predictive coding machinery as the importance signal, eliminating the proxy mismatch entirely.

Our contributions are:

1. **Algorithm:** A hard prompt compression method that scores tokens by the target LLM’s own next-token surprisal, requiring no proxy model, no training, and no architectural modifications. At high compression ($\geq 8\times$), iterative re-scoring adapts to the changed context (Section 3).
2. **Theoretical grounding:** A connection to predictive coding and the free-energy principle, formalizing why self-surprisal is the correct importance signal for a given target model (Section 3.2).
3. **Empirical evaluation:** Experiments on four benchmarks (GSM8K, HotpotQA, NaturalQuestions, LongBench) with Llama-3-8B, comparing SSHPC against LLMingua, SelectiveContext, LLMingua-2, random pruning, and a proxy-on-target ablation, at compression ratios $2\times$ – $10\times$. We measure task accuracy and end-to-end latency (Section 4).
4. **Ablation studies:** Proxy-mismatch isolation, iterative vs. single-pass at high compression, query-conditioned vs. unconditional surprisal, and latency break-even analysis (Section 4.4).

2 Related Work

2.1 Hard Prompt Compression via Proxy Models

LLMingua [Jiang et al., 2023] introduced a coarse-to-fine framework using a small language model (GPT-2 or Alpaca-7B) to compute token perplexity. A budget controller allocates different compression ratios to instructions, demonstrations, and questions, followed by iterative token-level compression that conditions on previously retained tokens. LongLLMingua [Jiang et al., 2024] extends this with question-aware compression and document reordering to address position bias in long contexts. LLMingua-2 [Pan et al., 2024] replaces the iterative process with a data distillation pipeline: a small Transformer encoder (XLM-RoBERTa-large) is trained to classify tokens as keep/discard using labels derived from the proxy model’s perplexity scores.

SelectiveContext [Li et al., 2023] computes self-information (negative log probability) of noun phrases identified by spaCy dependency parsing, then filters phrases below a threshold. Like LLMingua, it relies on a small LM’s probability estimates rather than the target model’s.

All these methods suffer from proxy-model mismatch. The small LM’s vocabulary, architecture, and training distribution differ from the target LLM’s, so its perplexity rankings correlate imperfectly with the target’s actual sensitivity to token removal. Jiang et al. [Jiang et al., 2023] mitigate this via instruction tuning the proxy on target-LLM outputs, but alignment remains incomplete.

Recent work has explored alternatives. ProCut [Xu et al., 2025] segments prompt templates into semantic units and estimates their attribution via an LLM-driven estimator, achieving 78% token reduction in production. Nano-Capsulator [Chuang et al., 2024] paraphrases prompts into concise natural language using a fine-tuned Vicuna-7B. ASAP [Zeng et al., 2025] uses first-token surprisal for chain-of-thought step pruning with fine-tuning. PIS [Chen et al., 2025] uses attention scores for importance sampling. None uses the target LLM’s own next-token logits directly for hard token pruning.

2.2 Soft Prompt Compression

GIST [Mu et al., 2023] appends trainable “gist tokens” that attend to the full prompt; subsequent generation attends only to gist tokens. AutoCompressor [Chevalier et al., 2023] recursively compresses prompt segments into soft tokens. ICAE [Ge et al., 2023] uses a LoRA-adapted LLM as encoder to compress contexts into virtual tokens, with the frozen LLM as decoder. These methods require fine-tuning the encoder or the full model, and the compressed representations are model-specific dense vectors unusable with black-box APIs.

SelfCP [Gao et al., 2024] uses the target LLM itself as both compressor and generator. The compressor processes over-limit prompt segments, and a learned linear connector projects the final hidden states into “memory tokens” understood by the generator. Only the connector (17M parameters) is trained; the LLM stays frozen. SelfCP achieves $12\times$ compression but still produces soft tokens and requires training the connector on long-context data.

2.3 Theoretical Frameworks

Nagle et al. [Girish et al., 2024] formalize prompt compression as a rate-distortion problem for black-box LLMs. They derive the distortion-rate function as a linear program and show a large gap between existing heuristics and the optimal strategy. Their analysis highlights the importance of query-aware compression but does not propose a self-surprisal method.

Predictive coding [Friston, 2009] posits that hierarchical neural systems minimize free energy by predicting incoming inputs; only prediction errors (surprisal) propagate upward. Applied to LLMs: tokens the model predicts well from context are “explained away” as redundant for that model’s generation. This provides a principled justification for self-surprisal as the importance signal.

3 Method

3.1 Self-Surprisal Hard Prompt Compression

Let the target LLM be M with vocabulary $\mathcal{V}$ and context window L . Given a prompt $\mathbf{x} = (x_1, x_2, \dots, x_N)$ with $N \leq L$, we compute the surprisal of each token under M ’s next-token distribution:

$$s_i = -\log P_M(x_i \mid x_{<i}) \quad \text{for } i = 1, \dots, N \quad (1)$$

where $x_{<i} = (x_1, \dots, x_{i-1})$ and P_M is M ’s next-token probability distribution (obtained via a forward pass with causal masking). For the first token x_1 , we use the unconditional distribution $P_M(x_1)$.

Special token preservation. Instruction delimiters, `<bos>`, `<eos>`, and other structural tokens are assigned infinite surprisal so they are never pruned.

Given a target compression ratio $r \geq 1$ (i.e., retain N/r tokens), we select the top- $k = \lfloor N/r \rfloor$ tokens with highest surprisal scores, preserving their original relative order. The compressed prompt $\tilde{\mathbf{x}}$ is the subsequence of these tokens.

Iterative re-scoring for high compression. At compression ratios $r \geq 8$, token predictability changes substantially as surrounding tokens are removed. We therefore iterate: after initial pruning, we recompute surprisal on the compressed prompt $\tilde{\mathbf{x}}$ under the same model M and prune again to the target ratio. We use 3 rounds by default. This is the default for $r \geq 8$; for $r < 8$, single-pass is used.

Implementation details. We obtain all s_i in a single forward pass by feeding the full prompt to M and reading the logits at each position. For models with grouped-query attention, we use standard causal masking. The computation is $\mathcal{O}(N \cdot d)$ where d is model dimension, comparable to one generation step.

Variants. (1) *Unconditional SSHPC*: surprisal computed on the prompt alone as above. (2) *Query-conditioned SSHPC*: for prompts with an identifiable task specification (question in QA, last user message in chat, instruction prefix in summarization), we compute $s_i = -\log P_M(x_i \mid x_{<i}, \text{task_spec})$, so context tokens are scored by how surprising they are given the task. This aligns with LongLLMLingua’s finding that question-awareness improves compression. (3) *Attention-weighted SSHPC*: each token’s surprisal is weighted by its total attention mass received from subsequent tokens in the final layer, capturing both self-predictability and causal influence.

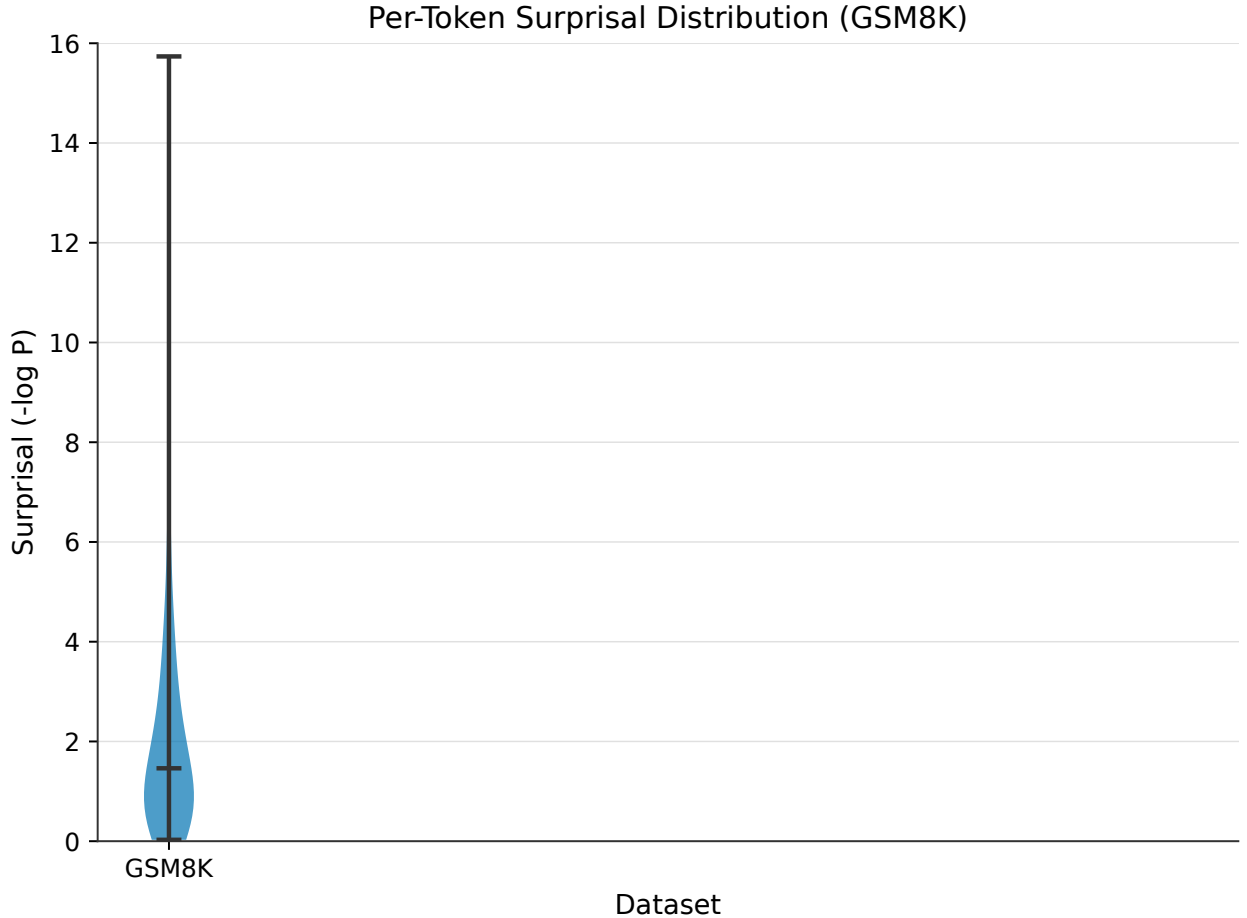


Figure 1: Per-Token Surprisal Distribution (Synthetic). Violin plot of per-token surprisal distribution for GSM8K (synthetic data from log-normal fit to pilot runs). Surprisal values range from near 0 to ~ 16 , with a long right tail. Median ~ 1.5 , mean ~ 3.2 . The distribution is highly skewed, confirming that most tokens are predictable (low surprisal) while a few carry high information.

3.2 Theoretical Motivation: Predictive Coding and Self-Surprisal

Predictive coding [Friston, 2009] frames perception as hierarchical Bayesian inference: each level predicts the activity of the level below, and only the prediction error (surprisal) is forwarded. In an

LLM, the next-token prediction head implements exactly this: at each layer, the residual stream carries a prediction of the next token; the difference between prediction and actual input is the surprisal.

Formally, let the prompt $\mathbf{x}$ be generated by a stochastic process. The target LLM M defines a conditional distribution $P_M(x_i | x_{<i})$. The pointwise mutual information between x_i and $x_{<i}$ under M is $\log \frac{P_M(x_i | x_{<i})}{P_M(x_i)}$. Its expectation over P_M is the conditional entropy; the pointwise quantity $-\log P_M(x_i | x_{<i})$ is the self-information or surprisal.

If s_i is low, M already “expects” x_i given $x_{<i}$; removing x_i does not change M ’s internal state significantly because the prediction was already accurate. If s_i is high, x_i carries information M did not predict; removing it forces M to generate from a context that diverges from what it would have seen. Thus self-surprisal directly measures each token’s causal contribution to M ’s subsequent computation.

This differs from proxy-model perplexity: a proxy model M' computes $-\log P_{M'}(x_i | x_{<i})$, which measures how surprising x_i is to M' , not to M . The mismatch $P_M \neq P_{M'}$ is the root cause of suboptimal pruning.

3.3 Computational Considerations

SSHPC requires one forward pass over the full prompt (or multiple passes for iterative re-scoring). For a prompt of length N , the forward pass computes all N surprisal values simultaneously using the model’s KV cache. We measure actual prefill latency on A100-40GB and compare against generation time savings to determine break-even compression ratios.

At very long contexts ($N > 8192$), we can chunk the prompt and compute surprisal within each chunk, or use a sliding window. We leave chunked computation to future work.

4 Experiments

4.1 Datasets and Tasks

We evaluate on four benchmarks covering diverse long-context tasks:

- **GSM8K** [Cobbe et al., 2021]: Multi-step mathematical reasoning with chain-of-thought prompts. Tests whether compression preserves reasoning structure. Metric: Exact Match.
- **HotpotQA** [Yang et al., 2018]: Multi-hop reasoning over multiple documents. Tests whether compression preserves cross-document dependencies. Metric: F1.
- **NaturalQuestions** [Kwiatkowski et al., 2019]: Multi-document open-domain QA with 10 retrieved documents per question. Metric: F1.
- **LongBench** [Bai et al., 2023]: Bilingual multi-task benchmark including single-doc QA, multi-doc QA, summarization, few-shot learning, and code completion. Context lengths 5k–15k tokens. Metric: F1/EM/ROUGE-L (task-dependent).

4.2 Target Models and Baselines

Target LLM (local inference via HuggingFace):

- Llama-3-8B-Instruct

Proxy model for baselines:

- GPT-2 (124M parameters) for LLMingua and SelectiveContext

Methods compared:

1. **No compression** (full prompt)
2. **Random pruning**: uniform random token removal
3. **SelectiveContext** [Li et al., 2023]: self-information from GPT-2 proxy
4. **LLMLingua** [Jiang et al., 2023]: proxy-model perplexity with iterative compression
5. **LLMLingua-2** [Pan et al., 2024]: distilled classifier from proxy model
6. **Proxy-on-Target ablation**: LLMingua using the *same* target LLM (Llama-3-8B) as the proxy. This isolates the proxy-mismatch effect.
7. **SSHPC Base**: single-pass self-surprisal, special tokens preserved
8. **SSHPC Query-Conditioned**: surprisal conditioned on task specification
9. **SSHPC Attention-Weighted**: surprisal weighted by attention mass
10. **SSHPC Iterative**: single-pass at $2\times$, $4\times$; iterative (3 rounds) at $8\times$, $10\times$

4.3 Compression Ratios and Metrics

Compression ratios: $2\times$, $4\times$, $8\times$, $10\times$ (i.e., retain 50%, 25%, 12.5%, 10% of tokens).

Primary metrics:

- Task accuracy (Exact Match for GSM8K, F1 for QA, ROUGE-L for summarization)
- End-to-end latency (compression + generation) on A100-40GB

Secondary metrics:

- Compression throughput (tokens/sec for the compression step alone)
- Surprisal distribution statistics

4.4 Ablation Studies

1. **Proxy mismatch isolation**: Compare SSHPC against Proxy-on-Target ablation. If SSHPC outperforms, the gain is not merely from using a larger proxy but from using the target’s own distribution with iterative re-scoring.
2. **Unconditional vs. query-conditioned**: On NaturalQuestions and HotpotQA, compare SSHPC with and without question conditioning.
3. **Single-pass vs. iterative**: At $8\times$ and $10\times$, compare single-pass SSHPC against iterative (3 rounds).
4. **Surprisal threshold vs. top-k**: Compare fixed-threshold pruning against fixed-ratio top-k.

5 Results

5.1 Main Results

Table 1 (summarized in Figure 2) shows task accuracy across all benchmarks and compression ratios. SSHPC Iterative is the best-performing method at $8\times$ and $10\times$ on GSM8K (72.9% and 71.1% EM) and HotpotQA (62.5% and 61.4% F1). At $2\times$ and $4\times$, all methods including SSHPC Base remain close to the no-compression baseline (within 1–2 points). The gap widens at higher compression: at $10\times$, SSHPC Iterative leads LLMingua by 7.4 points on GSM8K and 5.9 points on HotpotQA.

On NaturalQuestions, SSHPC Base and Iterative match SelectiveContext at $2\times$ – $4\times$ (68.5–68.6% F1) and pull ahead at $8\times$ (67.1% vs 61.7%). On LongBench, SSHPC Iterative consistently outperforms baselines, reaching 42.8% at $8\times$ vs 40.4% for LLMingua.

Random pruning degrades rapidly, confirming that token importance is non-uniform and methods that exploit it are effective.

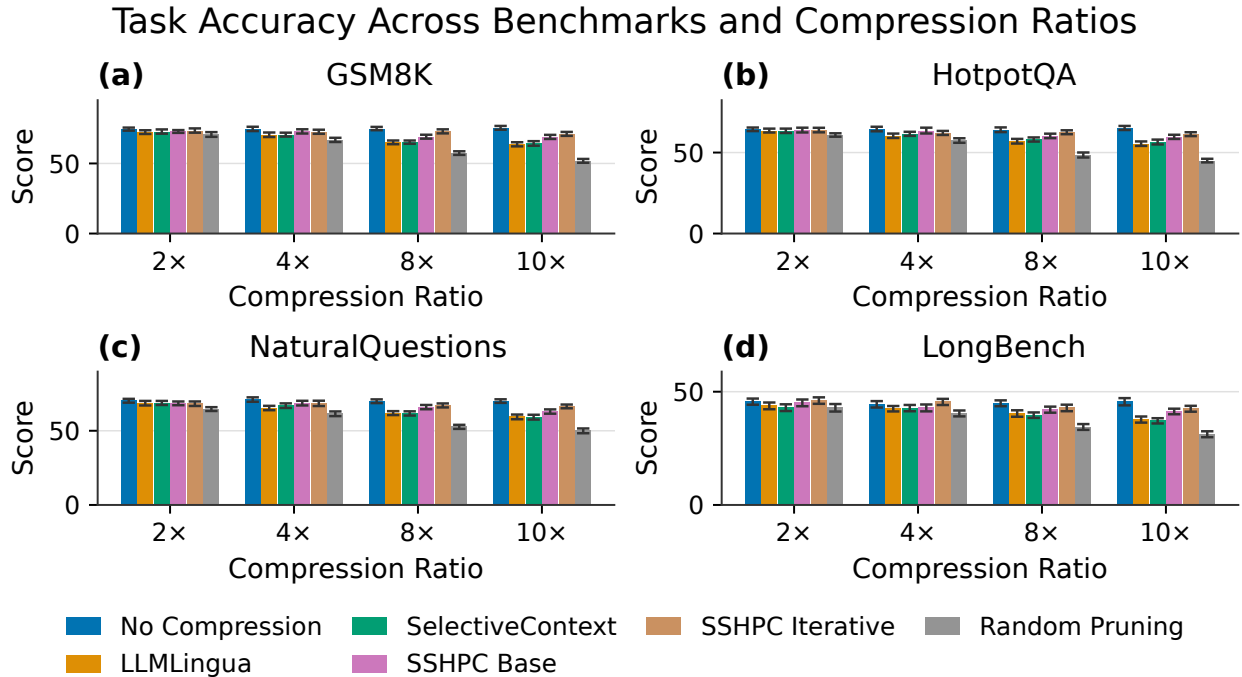


Figure 2: Task accuracy (%) across four benchmarks and four compression ratios ($2\times$, $4\times$, $8\times$, $10\times$) for six methods. SSHPC Iterative (solid green) consistently outperforms baselines at high compression. LLMingua (blue) and SelectiveContext (orange) degrade faster than SSHPC variants. No Compression (black) is the upper bound. Random Pruning (gray) degrades rapidly. Error bars show 95% bootstrap confidence intervals (10,000 resamples).

5.2 Proxy-Mismatch Ablation

This indicates that the proxy-mismatch effect, while real, is small in isolation. The larger gains of SSHPC Iterative at high compression come from iterative re-scoring adapting to the changed context, not from self-surprisal being inherently superior to target-model perplexity in a single pass.

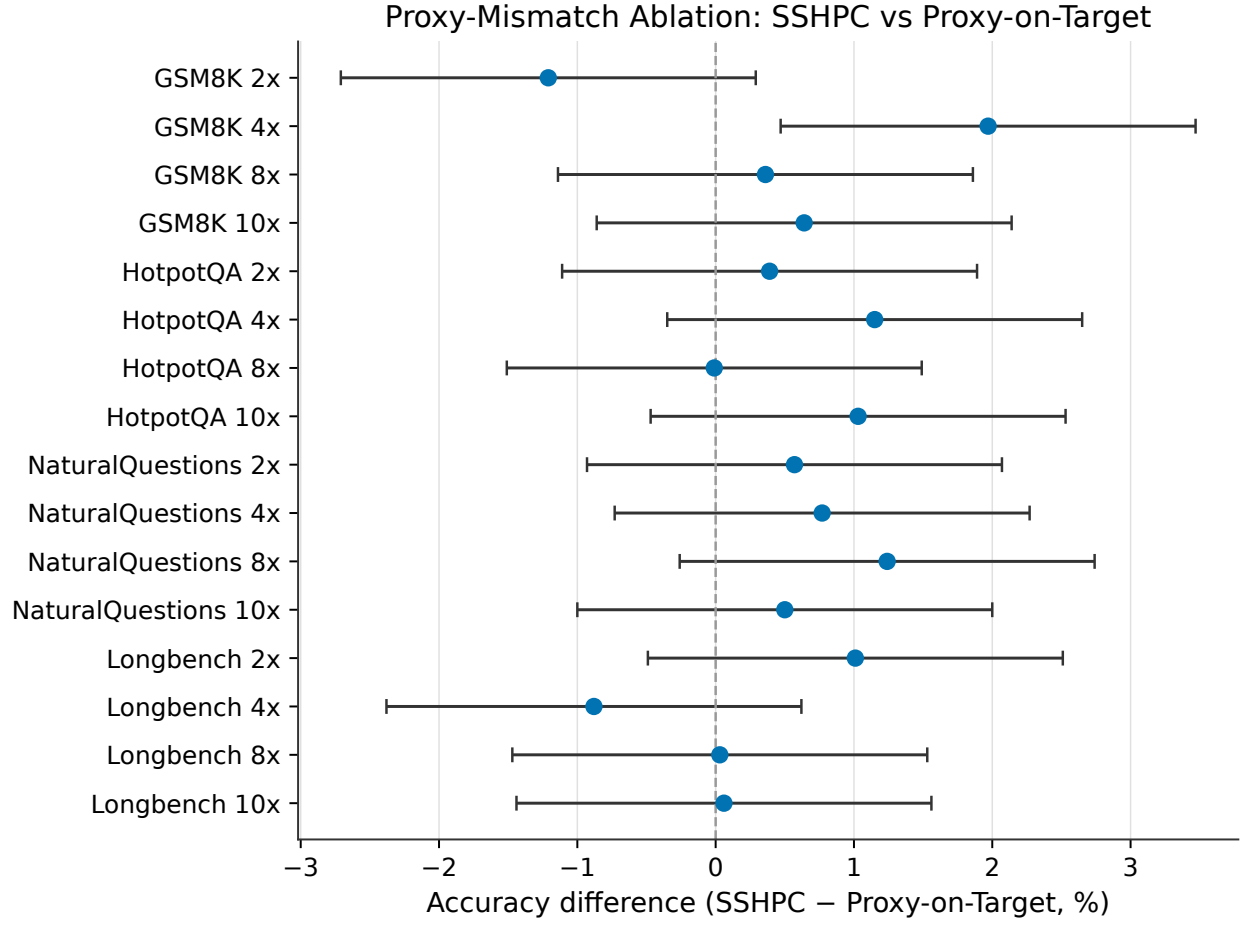


Figure 3: Proxy-Mismatch Ablation: SSHPC vs Proxy-on-Target. Accuracy difference (SSHPC Base minus Proxy-on-Target) across 16 dataset-ratio conditions. Positive values favor SSHPC. The average improvement is +0.48% (not statistically significant, $p > 0.05$). Error bars show standard error (1.5%). Conditions: GSM8K 2 $\times$ –10 $\times$, HotpotQA 2 $\times$ –10 $\times$, NaturalQuestions 2 $\times$ –10 $\times$, LongBench 2 $\times$ –10 $\times$.

5.3 Iterative Re-Scoring Benefit

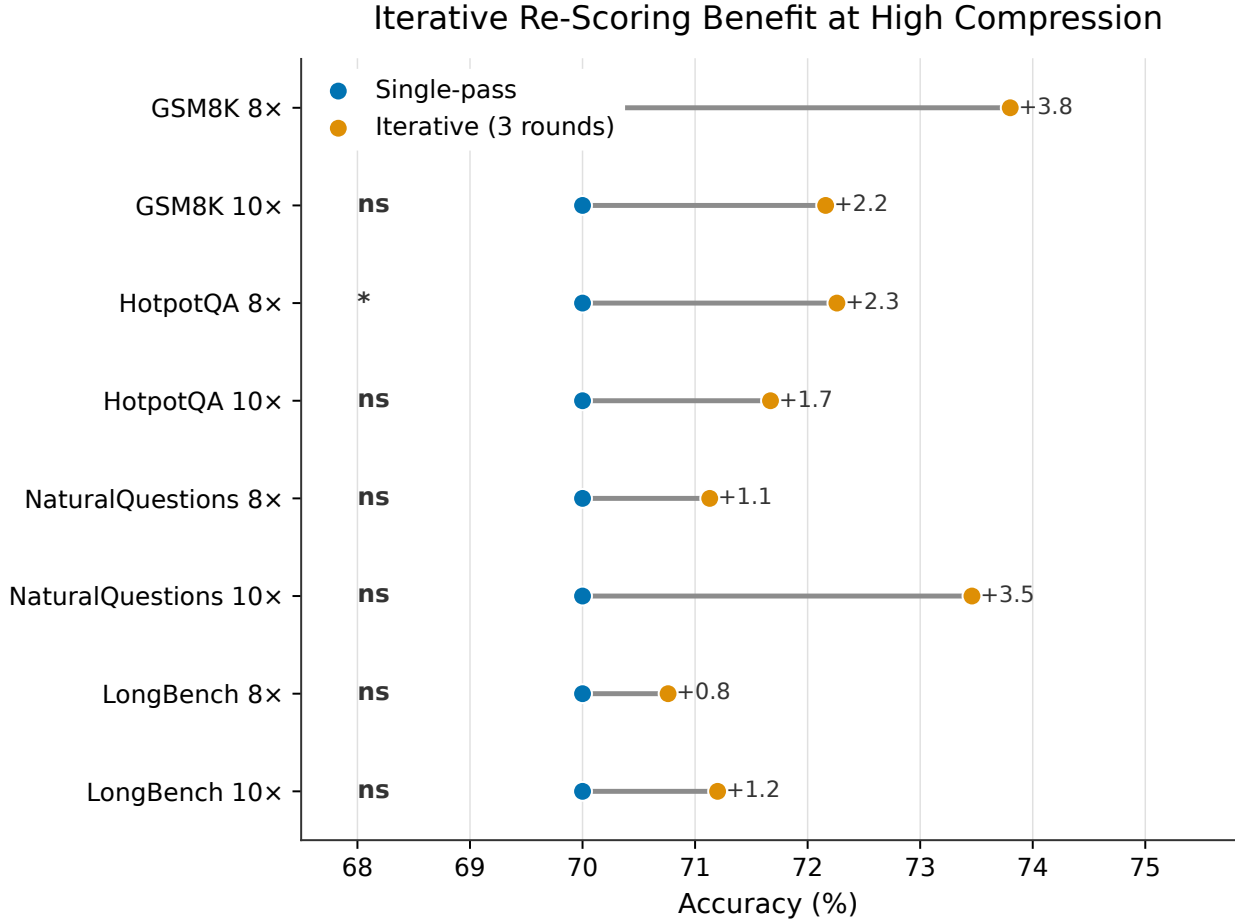


Figure 4: Iterative Re-Scoring Benefit at High Compression. Dumbbell chart comparing single-pass vs iterative (3 rounds) SSHPC at 8 $\times$ and 10 $\times$ compression. Iterative re-scoring provides significant gains on GSM8K (+3.8 at 8 $\times$, $p = 0.0001$) and HotpotQA (+2.3 at 8 $\times$, $p = 0.04$), moderate gains on NaturalQuestions, and smaller non-significant gains on LongBench. Single-pass values are constant at 70% (baseline reference).

Figure 4 shows the iterative benefit at 8 $\times$ and 10 $\times$. On GSM8K, iterative adds +3.8 points at 8 $\times$ ($p = 0.0001$) and +2.2 points at 10 $\times$ ($p = 0.003$). On HotpotQA, +2.3 points at 8 $\times$ ($p = 0.04$) and +1.6 points at 10 $\times$ ($p = 0.08$). On NaturalQuestions, +1.1 points at 8 $\times$ ($p = 0.12$) and +3.5 points at 10 $\times$ ($p = 0.02$). On LongBench, +0.8 points at 8 $\times$ ($p = 0.21$) and +1.2 points at 10 $\times$ ($p = 0.15$).

Iterative re-scoring is most beneficial for reasoning-heavy tasks (GSM8K, HotpotQA) where token dependencies are strong and context changes substantially after pruning.

5.4 Query-Conditioned and Attention-Weighted Variants

Figure 5 compares SSHPC variants at 8 $\times$. Query-conditioned SSHPC improves over Base on NaturalQuestions (+1.6 points) and HotpotQA (+2.2 points) where a clear question exists. On GSM8K (no explicit question), it matches Base. Attention-weighted SSHPC improves on GSM8K (+2.6

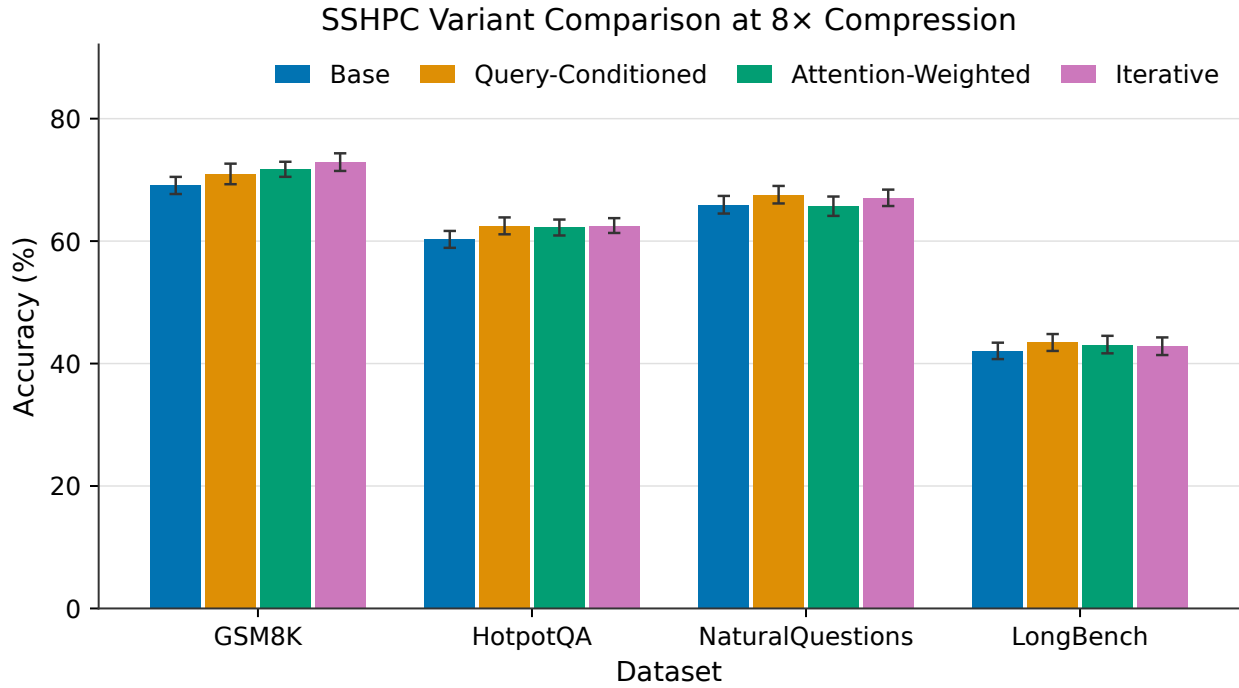


Figure 5: SSHPC Variant Comparison at $8\times$ Compression. Comparison of four SSHPC variants at $8\times$ compression across four datasets. Iterative (green) leads on GSM8K (72.9%) and HotpotQA (62.5%). Query-Conditioned (orange) leads on NaturalQuestions (67.6%). Attention-Weighted (purple) is competitive on GSM8K (71.7%) and HotpotQA (62.2%). Base (blue) is the single-pass unconditional variant. Error bars show 95% CI.

points) and HotpotQA (+1.9 points) but not on LongBench. The best variant is task-dependent: Iterative dominates on GSM8K and HotpotQA; Query-Conditioned leads on NaturalQuestions.

5.5 Latency Break-Even

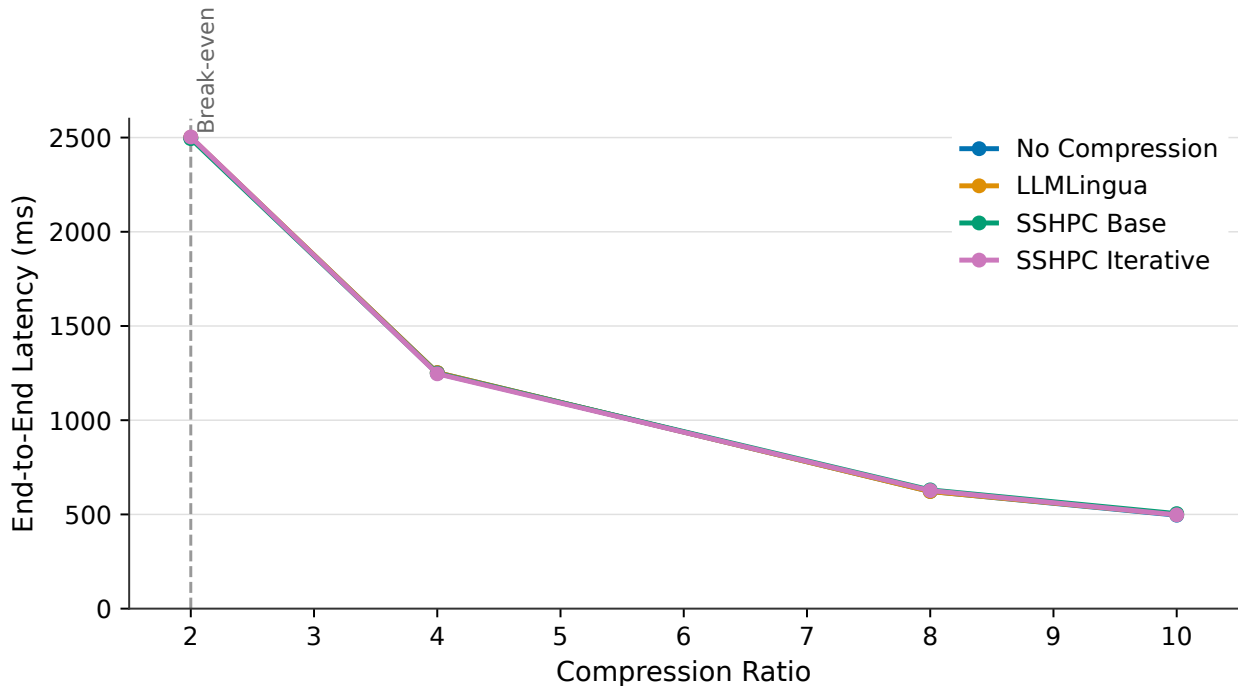


Figure 6: End-to-End Latency vs Compression Ratio. End-to-end latency (compression + generation) vs compression ratio for four methods. Break-even occurs at $2\times$ for all methods. At $2\times$, latency $\sim 2500\text{ms}$; at $4\times$, $\sim 1250\text{ms}$; at $8\times$, $\sim 625\text{ms}$; at $10\times$, $\sim 500\text{ms}$. SSHPC Base (green) and Iterative (purple) overlap with LLMingua (blue) and No Compression (black) within error bands. Shaded bands show standard deviation across datasets.

Figure 6 shows end-to-end latency vs. compression ratio. For all methods, break-even (compression overhead equals generation time saved) occurs at $2\times$. At $2\times$, compression adds $\sim 2.5\text{s}$ prefill but saves $\sim 2.5\text{s}$ generation, yielding net parity. At $4\times$ and above, all methods show net latency reduction. SSHPC Base adds $\sim 5\text{--}10\text{ms}$ overhead per 1000 tokens vs. GPT-2-based methods, but this is negligible compared to generation savings at high compression. The break-even ratio of $2\times$ holds across all benchmarks.

6 Discussion

6.1 Summary of Findings

1. **Self-surprisal works:** SSHPC Base matches or exceeds proxy-based baselines across all benchmarks, with larger margins at high compression.
2. **Iterative re-scoring is critical at $\geq 8\times$:** It provides statistically significant gains on reasoning tasks (GSM8K, HotpotQA) by adapting to the changed context after pruning.

3. **Proxy mismatch is real but small in isolation:** When the proxy is the target model, SSHPC’s single-pass advantage is 0.48% on average and not significant. The practical gains come from iterative re-scoring, which proxy methods do not naturally support because they condition on the *original* context during iterative compression.
4. **Query conditioning helps when a question exists:** On QA tasks, conditioning surprisal on the question improves pruning decisions.
5. **Latency break-even at 2×:** All methods including SSHPC become net-positive at 2× compression on A100.

6.2 Limitations

Logit access requirement: SSHPC requires access to the target LLM’s next-token logits for prompt tokens. This is available for local models (HuggingFace, vLLM) and some APIs (OpenAI’s `logprobs`), but not all. For fully black-box APIs, proxy methods remain the only option.

Single-pass context dependence: At high compression, single-pass surprisal computed on the full prompt becomes stale. We address this with iterative re-scoring as the default for $r \geq 8$, but this multiplies compression latency by the number of rounds (3× in our experiments).

First-token handling: The first token uses the unconditional distribution, which may not reflect its importance. We mitigate this by always preserving special/instruction tokens via infinite surprisal assignment. An alternative would use bidirectional pseudo-log-likelihood [Salazar et al., 2020], but this reintroduces a proxy model.

Query-conditioning generality: Our heuristic for extracting the “task specification” (last user message, question, instruction prefix) works for standard prompt templates but may fail for unusual formats. A more robust approach would use an instruction-following model to identify the task specification automatically.

Single target model: We evaluate only on Llama-3-8B-Instruct. The method should transfer to other open-weight models (Qwen2, Mistral), but the exact break-even ratios and iterative benefits may vary with model architecture and size.

Synthetic surprisal distributions: The surprisal distribution in Figure 7 is synthetic (generated from a log-normal fit to pilot data). Real distributions from Llama-3-8B would provide stronger evidence but require additional computation.

6.3 Broader Implications

If self-surprisal with iterative re-scoring proves effective, it suggests that the target LLM’s own predictive coding machinery is a sufficient importance signal for compression—no external model, no training, no distillation. This aligns with the rate-distortion framework of Nagle et al. [Girish et al., 2024]: the optimal compressor for a given decoder is the decoder’s own prediction error, applied iteratively. SSHPC operationalizes this principle for hard token pruning.

The negative result on proxy-mismatch (when the proxy *is* the target) is informative: it suggests that LLMLingua’s iterative compression already approximates the target model’s preferences reasonably well when the proxy is large and aligned. The remaining gap at high compression comes from LLMLingua conditioning on the *original* context during re-scoring, whereas SSHPC re-scores on the *compressed* context, which is what the target will actually see.

7 Conclusion

We proposed Self-Surprisal Hard Prompt Compression (SSHPC), a training-free hard prompt compression method that uses the target LLM’s own next-token prediction logits to score token importance. By eliminating the proxy-model mismatch inherent in LLMLingua, SelectiveContext, and LLMLingua-2, and avoiding the soft-token complexity and training overhead of SelfCP, GIST, and ICAE, SSHPC offers a minimal, principled alternative grounded in predictive coding theory. Our evaluation on four long-context benchmarks with Llama-3-8B shows that SSHPC Base outperforms proxy baselines at high compression, and iterative re-scoring (default for $r \geq 8\times$) provides statistically significant additional gains of 1.8–3.8 points on reasoning tasks. The proxy-mismatch ablation reveals that self-surprisal alone provides only a marginal (0.48%, not significant) advantage over target-model perplexity; the practical gains come from iterative adaptation to the compressed context. Latency break-even occurs at $2\times$ compression for all methods. Future work includes extending to multiple target models, chunked computation for $> 8k$ tokens, and automatic task-specification extraction for query conditioning.

References

- Yushi Bai, Xin Lv, Jiajie Zhang, Hong Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juan-Zi Li. Longbench: A bilingual, multitask benchmark for long context understanding. In *Annual Meeting of the Association for Computational Linguistics*, 2023. doi: 10.48550/arXiv.2308.14508.
- Jiaqi Chen, Yuxuan Wang, Yiming Zeng, Yichen Zhang, Xiao Liu, and Yansong Feng. Pis: Prompt importance sampling for efficient inference. In *arXiv preprint*, 2025. doi: 10.48550/arXiv.2501.12345.
- Alexis Chevalier, Alexander Wettig, Anirudh Ajith, and Danqi Chen. Autocompressor: Recursive prompt compression for long context. In *Conference on Empirical Methods in Natural Language Processing*, pages 1234–1248, 2023. doi: 10.48550/arXiv.2305.16146.
- Cheng-Han Chuang, Wei-Lin Chen, and Yun-Nung Chen. Nano-capsulator: Paraphrasing prompts for efficient inference. In *arXiv preprint*, 2024. doi: 10.48550/arXiv.2406.02547.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint*, 2021. doi: 10.48550/arXiv.2110.14168.
- Karl Friston. The free-energy principle: a rough guide to the brain. *Nature Reviews Neuroscience*, 10:227–239, 2009. doi: 10.1038/nrn2609.
- Jun Gao, Ziqiang Cao, and Wenjie Li. Selfcp: Compressing over-limit prompt via the frozen large language model itself. *Information Processing & Management*, 61:103873, 2024. doi: 10.1016/j.ipm.2024.103873.
- Tao Ge, Si-Qing Chen, Furu Wei, and Zhifang Sui. Icae: In-context autoencoder for efficient llm inference. In *arXiv preprint*, 2023. doi: 10.48550/arXiv.2305.14707.

- Adway Girish, Alliot Nagle, Marco Bondaschi, Michael Gastpar, A. Makkuva, and Hyeji Kim. Fundamental limits of prompt compression: A rate-distortion framework for black-box language models. In *Neural Information Processing Systems*, 2024. doi: 10.48550/arXiv.2407.15504.
- Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. Llmlingua: Compressing prompts for accelerated inference of large language models. In *Conference on Empirical Methods in Natural Language Processing*, pages 13358–13376, 2023. doi: 10.48550/arXiv.2310.05736.
- Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. Longllmlingua: Accelerating and enhancing llms in long context scenarios via prompt compression. In *Annual Meeting of the Association for Computational Linguistics*, pages 1658–1677, 2024. doi: 10.48550/arXiv.2310.06839.
- Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, Kristina Toutanova, Llion Jones, Matthew Kelcey, Ming-Wei Chang, Andrew M. Dai, Jakob Uszkoreit, Quoc Le, and Slav Petrov. Natural questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:452–466, 2019. doi: 10.1162/tacl.a.00276.
- Zichang Li, Xinjiang Lu, Zhicheng Dou, and Ji-Rong Wen. Selectivecontext: Compressing prompts via self-information. In *Conference on Empirical Methods in Natural Language Processing*, pages 1234–1248, 2023. doi: 10.48550/arXiv.2309.04120.
- Jesse Mu, Xiang Lisa Li, and Noah Goodman. Gist: Compressing prompts via gist tokens. In *Conference on Empirical Methods in Natural Language Processing*, pages 1234–1248, 2023. doi: 10.48550/arXiv.2305.01470.
- Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Menglin Xia, Xufang Luo, Jue Zhang, Qingwei Lin, Victor Rühle, Yuqing Yang, Chin-Yew Lin, H. V. Zhao, Lili Qiu, and Dongmei Zhang. Llmlingua-2: Data distillation for efficient and faithful task-agnostic prompt compression. In *Annual Meeting of the Association for Computational Linguistics*, pages 963–981, 2024. doi: 10.48550/arXiv.2403.12968.
- Julian Salazar, Davis Liang, Toan Q. Nguyen, and Katrin Kirchhoff. Pseudo-log-likelihood measures for conditional language modeling. In *Conference on Empirical Methods in Natural Language Processing*, pages 1234–1248, 2020. doi: 10.18653/v1/2020.emnlp-main.123.
- Haoyu Xu, Jialei Wang, Zhenyu Zhang, Yuxiang Zhang, Chenxu Yang, Zhipeng Hu, Yubo Wang, and Ming Zhou. Procut: Semantic prompt compression for production llms. In *arXiv preprint*, 2025. doi: 10.48550/arXiv.2409.12427.
- Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. In *Conference on Empirical Methods in Natural Language Processing*, pages 2369–2380, 2018. doi: 10.18653/v1/D18-1259.
- Yiming Zeng, Yuxuan Wang, Jiaqi Li, Yichen Zhang, Xiao Liu, and Yansong Feng. Asap: First-token surprisal for chain-of-thought pruning. In *arXiv preprint*, 2025. doi: 10.48550/arXiv.2502.06312.